

o-level-computing-teaching-and-learning-syllabus (Traduzido)

Traduzido em 29 de dezembro de 2025

COMPUTAÇÃO NÍVEL-O

PROGRAMA DE ESTUDOS

Ensino Secundário Superior

Implementação a partir da

Turma do Terceiro Ano do Ensino Secundário de 2017

© 2016 Divisão de Planeamento e Desenvolvimento Curricular.

Esta publicação não está à venda. Todos os direitos reservados. Nenhuma parte desta publicação pode ser reproduzida sem a permissão prévia do

Ministério da Educação, Singapura.

Ministério da Educação

SINGAPURA

ÍNDICE

1. INTRODUÇÃO Página

- Visão Geral 4
- Importância da Disciplina 5
- Estrutura da Educação em Computação 5
- Grandes Ideias em Computação 7
- Objetivos do Programa 8

2. CONTEÚDO

- Visão Geral do Programa 10
- Módulo I: Dados e Informações 11
- Módulo II: Sistemas e Comunicações 14
- Módulo III: Abstração e Algoritmos 16
- Módulo IV: Programação 18

3. PEDAGOGIA

- Aprendizagem Aplicada 21
- Aprender Fazendo 23

4. AVALIAÇÃO

- Avaliação Escolar 25
- Exame Nacional 26

Anexo A: Referência Rápida para Python 29

Anexo B: Referência Rápida para Fluxogramas 31

Anexo C: Glossário de Termos 33

SEÇÃO 1:

INTRODUÇÃO

Visão Geral
Importância da Disciplina
Estrutura da Educação em Computação
Grandes Ideias em Computação
Objetivos do Programa de Estudos

3

1. INTRODUÇÃO

Visão Geral

Nas últimas décadas, o avanço da tecnologia da computação tem sido a principal força motriz por trás de muitas das mudanças observadas em vários aspectos de nossas vidas. Além desse impacto em nossa vida diária como usuários de tecnologia, a Computação tornou-se tão arraigada nos campos de Ciência, Tecnologia, Engenharia e Matemática (STEM) que transformou as próprias práticas dessas disciplinas.

A análise de dados está transformando o futuro dos negócios e da pesquisa em diversas áreas. Precisaremos de maior capacidade computacional e de pessoas com consideráveis habilidades de pensamento computacional para automatizar e otimizar processos, trabalhar com dados (por exemplo, texto e imagens) para discernir tendências e tirar inferências, desenvolver hipóteses e inovar em campos como medicina, ciência e engenharia para resolver problemas complexos.

Profissionais de finanças, varejo e manufatura precisarão de habilidades de pensamento computacional para serem competitivos em suas áreas. Por exemplo, será um trunfo adquirir a capacidade de usar um computador para analisar dados e prever tendências de mercado e de negócios, entender as condições econômicas e construir relacionamentos participativos com clientes no ambiente digital. Os alunos que estudam Computação no nível O-level adquirirão uma base mais sólida em pensamento computacional que será útil para seus estudos e carreiras futuras.

Propósito e Filosofia

O cenário global da educação em computação está experimentando uma ênfase renovada na programação (codificação) e nas habilidades computacionais fundamentais que prepararão os alunos para prosperar em um mundo e local de trabalho mais conectados digitalmente. A infinidade de aplicativos em dispositivos móveis e de computação mudou a forma como vivemos, aprendemos e trabalhamos. O impacto é sentido em todas as faixas etárias. Para reduzir a divisão digital, vários esquemas e iniciativas governamentais foram lançados para atingir a população, de modo que mais pessoas possam aproveitar a tecnologia de forma eficaz para melhorar suas vidas e contribuir para a sociedade.

Enquanto o programa de TIC de linha de base atual equipa todos os alunos com habilidades básicas de alfabetização digital para serem usuários informados da tecnologia, o currículo de

Computação O-level visa aumentar o interesse e a competência dos alunos em conceitos e habilidades mais avançadas. Isso os equipará com a base necessária para continuar com cursos de computação pós-ensino médio. Um objetivo secundário é incentivar os alunos a considerar carreiras em tecnologia e sistemas de computação ou como programadores qualificados, desenvolvedores de sistemas e engenheiros de software. Nossa aspiração é que esse grupo de alunos com paixão por Computação eventualmente utilize seu talento para resolver problemas complexos ou criar novas propostas de valor na sociedade por meio da tecnologia.

4

Importância da Disciplina

O valor da computação surge do fato de que ela tem a capacidade de integrar o uso de software e hardware para criar novos artefatos, resolver/abordar problemas do mundo real e executar tarefas. A computação está fundamentada em (i) pensamento computacional e (ii) pensamento sistêmico; e permite (iii) exploração e inovação. Assim, o Pensamento Computacional e o Pensamento Sistêmico formam os dois braços na dimensão do Computador como Ciência em nosso framework de educação em computação (mostrado na Figura 1.1). As outras duas dimensões sobre o Computador como Ferramenta e o Computador na Sociedade também abordam áreas importantes na Computação e são ensinadas para que os alunos sejam capazes de desenvolver suas ideias e criatividade através do uso de tecnologia da informação e comunicação (TIC) para contribuir com a sociedade.

Pensamento Computacional

O pensamento computacional é um processo de pensamento que envolve raciocínio formal, pensamento lógico e algorítmico, e a reformulação de um problema para que uma solução baseada em computador seja viável.

Na disciplina de Computação de nível O, os alunos são ensinados a passar por um processo sistemático de pensamento ao resolver problemas (abstração), formular passos para soluções (pensamento algorítmico) e escrever programas de computador (programação/codificação) para produzir as soluções. Assim, os alunos desenvolvem habilidades de pensamento computacional ao analisar situações problema e reformulá-las em problemas com soluções computacionais. Eles também desenvolvem o pensamento computacional ao codificar soluções de programação eficazes e eficientes. Alunos com habilidades de pensamento computacional serão capazes de aplicar suas habilidades em outras disciplinas e estarão mais aptos a resolver problemas do mundo real nessas disciplinas. Munidos de habilidades de pensamento computacional, nossos alunos terão uma vantagem competitiva no cenário cada vez mais digital.

A promoção do pensamento computacional nos alunos também se destacou predominantemente nos programas de estudos oferecidos nos países pesquisados (por exemplo, Reino Unido, EUA, Austrália e Hong Kong). O pensamento computacional e o estudo da Computação

permitem aos alunos entender o mundo digital de uma forma mais profunda; assim como a física permite aos alunos entender melhor o mundo físico e o estudo da biologia permite uma melhor compreensão do mundo biológico.

Estrutura da Educação em Computação

A estrutura da educação em computação compreende três dimensões:

- Computador como Ciência
- Computador como Ferramenta
- Computador na Sociedade

Computador como Ciência

A dimensão do Computador como Ciência investiga o aspecto científico da ciência da computação, focando nos componentes centrais do pensamento computacional e sistêmico.

5

O pensamento computacional desenvolve as habilidades dos alunos na resolução de problemas por meio do pensamento algorítmico e do design. A aquisição de habilidades em linguagem de programação geralmente faz parte dessa área de aprendizado. O pensamento computacional, conforme definido por Jeannette M. Wing¹, é uma forma de as pessoas resolverem problemas e que não se trata de tentar fazer com que as pessoas pensem como computadores². Isso geralmente envolve processos de pensamento e resolução de problemas para reformular uma tarefa aparentemente difícil em uma que sabemos resolver. Assim, o pensamento computacional, em sua opinião, é uma habilidade fundamental para todos, não apenas para cientistas da computação.

O pensamento sistêmico desenvolve os alunos no design e criação de sistemas e soluções através de processos de definição de problemas, análise de sistemas e design de sistemas.

- Simulação
- Programação
- Estruturas de dados
- Bancos de dados
- Abstração
- Pensamento
algorítmico
- Matemática
discreta
- Hardware de
computador
- Resolução de problemas
dispositivos de TIC
- Montagem e manutenção
- Pensamento

Computacional
Uso do
Computador
FERRAMENTA
CIÊNCIA
SOCIEDADE

- Análise e Design de Sistemas
- Sistemas de Pensamento
- Engenharia de Software
- Redes e comunicações
- Uso Seguro e Responsável
- Gerenciamento de projetos
- Robótica

- Animação
- Edição de imagem
- Planilhas
- Processamento de texto
- Ferramentas de mídia

Uso de
Aplicativos

- Ferramentas de apresentação

Competências do
Século XXI

CAD/CAM
Design de página
web

- Jogos
- Pensamento Crítico e Inventivo
 - Habilidades de Informação e Comunicação
- Uso Ético e

Legal

- Vírus Vício

A internet

- Segurança

Segurança do computador

Mídias sociais • Dados

armazenamento

- Alfabetização Cívica,

Consciência Global

e Habilidades

Interculturais

- Propriedade

intelectual

- Privacidade de dados

Crimes cibernéticos

- Etiqueta na rede

- Ética

Figura 1.1: Estrutura da Educação em Computação (O anel externo são exemplos de tópicos que poderiam ser estudados para as três dimensões mostradas no anel interno. O anel do meio mostra os componentes de cada dimensão.)

1 Jeannette M. Wing é Professora President's de Ciência da Computação e chefe do Departamento de Ciência da Computação na Carnegie Mellon University.

2 J. M. Wing. Pensamento Computacional. Communications of the Association for Computing Machinery (ACM), Março de 2006, Vol. 49(3).

6

Computador como Ferramenta

A dimensão do Computador como Ferramenta observa principalmente o aspecto utilitário da computação e TICs. No centro disso estão o uso do computador e o uso de aplicativos de computador. O uso do computador expõe os alunos ao hardware, à tecnologia e aos dispositivos e periféricos relacionados que abrem caminhos para o trabalho, o lazer e a vida.

O uso de aplicativos de computador foca no domínio das ferramentas de produtividade, comunicação e criatividade para completar tarefas com propósitos específicos. Exemplos comuns incluem processamento de texto, planilhas, gráficos, e-mails, animação e design web.

Computador na Sociedade

Esta dimensão foca principalmente nas questões éticas, legais e de segurança relacionadas ao

uso de computadores e TIC na sociedade. Questões comumente associadas a esta dimensão incluem segurança na internet, propriedade intelectual, vício em computador e privacidade de dados.

A inclusão do componente de Competências do Século XXI reflete o impacto da tecnologia no tipo de habilidades necessárias no local de trabalho do futuro. As Competências do Século XXI relevantes para a área de TIC incluem a capacidade de trabalhar colaborativamente, produzir trabalho criativo e ser autodirigido na aprendizagem.

Grandes Ideias em Computação

O US College Board e a National Science Foundation (2010)³ focaram nas práticas de pensamento computacional para currículos de CS do ensino médio, baseados em sete "grandes ideias" da Computação. As "grandes ideias" são:

- (a) A computação é uma atividade humana criativa.
- (b) A abstração reduz informações e detalhes para focar em conceitos relevantes para entender e resolver problemas.
- (c) Dados e informações facilitam a criação de conhecimento.
- (d) Algoritmos são ferramentas para desenvolver e expressar soluções para problemas computacionais.
- (e) A programação é um processo criativo que produz artefatos computacionais.
- (f) Dispositivos digitais, sistemas e as redes que os interligam capacitam e promovem abordagens computacionais para resolver problemas.
- (g) A computação permite a inovação em outros campos, incluindo ciência, ciências sociais, humanidades, artes, medicina, engenharia e negócios.

Além disso, o programa de graduação "Ways of Thinking/Ways of Doing" da Universidade de Stanford descreve o raciocínio formal como uma forma de pensar que é dedutivo rigoroso (lógico e analítico). Essa forma de pensar sustenta as habilidades de tomada de decisão e análise que são aplicadas em muitos campos, incluindo a Computação.

³ AP Computer Science: Principles – Big Ideas, Key Concepts and Supporting Concepts (2010, The College Board e National Science Foundation).

O estudo do programa de graduação em Stanford: Ways of Thinking aborda a abordagem baseada em capacidade para promover amplitude em Eletivas de Educação Geral.

7

Objetivos do Programa de Estudos

O programa de estudos visa fornecer aos alunos a base para continuar com estudos futuros em computação e habilidades para participar de um ambiente tecnológico em rápida mudança, de modo que os conceitos e habilidades aprendidos também sejam aplicáveis em outros campos que exigem computação. O curso de dois anos no ensino médio superior visa capacitar os

alunos a:

1. Aplicar raciocínio lógico e pensamento algorítmico na análise de problemas e no desenvolvimento de soluções;
2. Desenvolver programas simples através do uso de linguagens de programação apropriadas;
3. Compreender como e onde a tecnologia da informação e comunicação está presente no dia a dia;
4. Compreender e explicar as questões éticas, sociais e econômicas associadas ao uso da TIC.

Os alunos aprenderão como os dados podem ser gerenciados e processados usando funções e fórmulas em um aplicativo de planilha para produzir informações e embasar a tomada de decisões; como os sistemas digitais funcionam e como esse conhecimento pode ser usado para criar casas seguras e redes protegidas; e como usar o computador de forma eficiente e eficaz para codificar soluções para situações problema. Detalhes sobre o que os alunos aprenderão estão na seção seguinte.

u O pensamento algorítmico é uma forma de chegar a uma solução através de vários passos. Em vez de uma única resposta, um conjunto de instruções ou regras é desenvolvido, que, se seguido precisamente, leva às respostas.

v Tecnologia da Informação e Comunicação (TIC) refere-se amplamente a dispositivos de computação, software e outros hardwares.

8

SEÇÃO 2: CONTEÚDO

Visão Geral do Programa

Módulo I: Dados e Informações

Módulo II: Sistemas e Comunicações

Módulo III: Abstração e Algoritmos

Módulo IV: Programação

9

2. CONTEÚDO

Visão Geral do Programa

Este programa de estudos compreende quatro módulos para cobrir cinco áreas comuns de conceitos e habilidades da ciência da computação. O estudo é realizado nos níveis do ensino

médio superior por dois anos. Os quatro módulos e as unidades de estudo para cada módulo estão listados com detalhes nas páginas seguintes. As escolas são encorajadas a ter, nos níveis do ensino médio inferior, um programa de ciência da computação que forneça aos alunos uma base sólida para cursar a disciplina nos níveis do ensino médio superior.

Os quatro módulos são:

Módulo I – Dados e Informações

- Gerenciamento de Dados
- Representação de Dados
- Questões Éticas, Sociais e Econômicas

Módulo II – Sistemas e Comunicações

- Arquitetura de Computadores
- Comunicações de Dados

Módulo III – Abstração e Algoritmos

- Análise de Problemas
- Projeto de Algoritmos

Módulo IV – Programação

- Desenvolvimento de Programas
- Teste de Programas

10

Módulo I: Dados e Informações

Este módulo trata do manuseio e processamento de dados em sistemas computacionais, bem como da necessidade de ser ético ao lidar com dados. Os alunos devem ser capazes de identificar diferentes tipos de dados, entender e explicar para que servem os dados, e explicar como os dados são representados ou organizados para processamento e saída, com referência a um problema dado. Os alunos se tornarão mais conscientes das questões éticas em relação aos dados, incluindo a privacidade dos dados. Há três unidades de estudo neste módulo:

1.1 Gerenciamento de Dados

1.2 Representação de Dados

1.3 Questões Éticas, Sociais e Econômicas

1.1 Gerenciamento de Dados

Esta unidade de estudo enfatiza o raciocínio lógico através da análise de dados, processamento de dados e representações visuais de dados. Os alunos utilizarão o software aplicativo de planilha em atividades práticas para aprimorar o aprendizado de funções e fórmulas para calcular e processar dados.

Ref. Resultado de Aprendizagem

Os alunos devem ser capazes de

1.1.1 Tabular dados sob títulos de coluna apropriados (ou seja, nomes de campos de dados) e tipos de dados (por exemplo, numérico, texto e data).

Excluir: plotagem de gráficos

1.1.2 Usar funções matemáticas, estatísticas e financeiras em contextos apropriados, incluindo análise de dados "e se" (what-if), para:

- encontrar o total de uma lista de números, a média de uma lista de números, min/max de uma lista de números, raiz quadrada de um número, juros simples, resto após a divisão de números;
- arredondar valores;
- randomizar valores;
- converter dados de um tipo para outro (por exemplo, obter valores inteiros de valores decimais); e
- contar o número de itens de dados.

A lista de 41 funções examináveis é:

TODAY, LEN, MID, LEFT, CEILING, FLOOR, MAX, MIN, IF, COUNT, COUNTA, COUNTIF, COUNTBLANK, POWER, SQRT, ROUND, RANDBETWEEN, RAND, SUM, SUMIF, MOD, MODE.SNGL, MEDIAN, AVERAGE, SMALL, LARGE, PERCENTILE, QUARTILE, STDEV, VAR, AND, OR, NOT, HLOOKUP, VLOOKUP, PV, FV, RATE, PMT, IPMT, PPMT.

Os alunos não devem ser penalizados por usar funções equivalentes não listadas.

11

Ref. Resultado de Aprendizagem

Os alunos devem ser capazes de

1.1.3 Compreender e usar declarações condicionais (simples e aninhadas): COUNTIF e IF com operadores relacionais e booleanos como AND, NOT e OR.

Inclui: formatação condicional

1.1.4 Usar funções de forma eficaz para pesquisar dados em linhas ou colunas (pesquisas horizontais e verticais de tabela) em uma lista ou tabela para processamento de dados.

1.2 Representação de Dados

Esta unidade de estudo explica como os dados são representados internamente como números binários, como a conversão de dados de um sistema numérico para outro é feita, e onde esses sistemas numéricos são usados. Os sistemas numéricos abordados aqui são o sistema numérico binário, o sistema numérico decimal e o sistema numérico hexadecimal.

Ref. Resultado de Aprendizagem

Os alunos devem ser capazes de

1.2.1 Representar números inteiros positivos em formato binário.

1.2.2 Converter números inteiros positivos de um sistema numérico para outro – binário, decimal e hexadecimal; e descrever a técnica utilizada.

1.2.3 Descrever situações em que os sistemas numéricos são usados, como códigos ASCII, IP (Protocolo de Internet) e endereços MAC (Media Access Control), e códigos RGB.

12

1.3 Questões Éticas, Sociais e Econômicas

Esta unidade de estudo abrange o uso ético e a segurança de dados em aplicativos de desktop ou aplicativos em redes como a internet. São discutidas questões sociais e econômicas decorrentes do uso de tecnologias de computação em ambientes de trabalho.

Ref. Resultado de Aprendizagem

Os alunos devem ser capazes de

1.3.1 Compreender como os dados podem ser mantidos seguros contra danos acidentais devido à corrupção de dados ou erros humanos e ações maliciosas, como visualização, exclusão, cópia e corrupção não autorizadas ou malware.

1.3.2 Compreender os efeitos de ataques cibernéticos como spam, spyware, cookies, phishing e pharming e o uso de medidas de segurança como hardware e software apropriados.

1.3.3 Descrever questões éticas relacionadas ao uso de computadores, redes públicas/privadas, software gratuito, shareware e cursos abertos; e o compartilhamento de informações.

1.3.4 Comparar os impactos sociais e econômicos positivos e negativos da tecnologia na educação, comunicação (por exemplo, via aplicativos móveis e mídias sociais), finanças, medicina/saúde, transporte e entretenimento; e explicar plágio e pirataria de software.

13

Módulo II: Sistemas e Comunicações

Este módulo trata de sistemas que envolvem tecnologia de computador e dispositivos de computação. Os alunos aprenderão as inter-relações entre as partes e o todo de um sistema; assim como as funções das partes dos sistemas para permitir a comunicação entre humanos e dispositivos de computação (máquina), máquina e máquina, e dentro de uma máquina. Há duas unidades de estudo:

2.1 Arquitetura de Computadores

2.2 Comunicações de Dados

2.1 Arquitetura de Computadores

Esta unidade de estudo abrange a arquitetura básica do computador e os componentes de uma rede de computadores. Os alunos compreenderão o propósito do hardware e software necessários em um sistema ou rede de computadores, mas não são obrigados a saber como eles funcionam tecnicamente. Os alunos terão a oportunidade de trabalhar com componentes de hardware e compreender o uso integrativo de hardware e software através do aprender fazendo.

Ref Resultado de aprendizado

Os alunos devem ser capazes de

2.1.1 Descrever a arquitetura básica de um computador com referência a:

- Processador do computador
- Memória
- Barramentos de dados e endereços
- Entrada (por exemplo, dados e instruções) e saída (por exemplo, resultados intermediários e finais do processamento)
- Armazenamento externo

Exclui: Ciclo de busca e execução

2.1.2 Reconhecer uma porta lógica a partir de sua tabela verdade e avaliar declarações Booleanas por meio de tabelas verdade.

2.1.3 Construir as tabelas verdade para circuitos lógicos dados (máximo 3 entradas).

2.1.4 Projetar e construir circuitos lógicos simples usando AND, OR, NOT, NAND e NOR.

14

2.2 Comunicações de Dados

Esta unidade de estudo utiliza redes como contexto para a compreensão do compartilhamento de recursos e comunicações de dados. Os alunos devem ter uma compreensão geral de como os dados são transmitidos e da necessidade de verificar a precisão e a segurança dos dados nas transmissões.

Ref Resultado de Aprendizagem

Os alunos devem ser capazes de

2.2.1 Identificar os diferentes dispositivos de rede: roteador, bridge e placa de interface de rede; e explicar suas funções.

2.2.2 Descrever a diferença entre redes com fio e sem fio e explicar os fatores que determinarão o uso de cada tipo de rede.

Inclui: descrições de LAN/MAN/WAN

2.2.3 Descrever os componentes para uma rede doméstica simples e projetar uma rede doméstica simples.

2.2.4 Comparar e contrastar estratégias de rede cliente-servidor e ponto a ponto com ênfase em:

- Propósito
- Função
- Organização
- Largura de banda

Inclui: princípios de topologia de redes em barramento, anel e estrela

2.2.5 Explicar o uso de paridade e checksums na transmissão de dados.

15

Módulo III: Abstração e Algoritmos

Este módulo trata da resolução de problemas e de como um problema pode ser resolvido dividindo-o em partes menores e gerenciáveis, e resolvendo todas as partes menores. Um algoritmo descreve uma solução para o problema que é independente de uma linguagem de programação e pode ser apresentado em pseudo-código (onde as estruturas do programa serão mais pronunciadas) ou diagramaticamente (fluxograma). Os alunos devem ser capazes de saber a diferença entre pseudo-código e fluxograma. Há duas unidades de estudo:

3.1 Análise de Problemas

3.2 Design de Algoritmos

3.1 Análise de Problemas

Esta unidade de estudo abrange a interpretação e análise de problemas. Os alunos aprenderão como abordar a resolução de problemas de forma sistemática. Eles aprenderão estratégias de resolução de problemas, como dividir um problema em suas partes e resolvê-lo resolvendo primeiro as partes individuais. Terão oportunidades de reforçar sua compreensão através de atividades práticas na resolução de problemas simples do mundo real.

Ref. Resultado de Aprendizagem

Os alunos devem ser capazes de

3.1.1 Definir e declarar um problema:

- Identificando as entradas necessárias
- Identificando as saídas necessárias
- Declarando os processos necessários

Exemplos:

- Negócios: produzir uma lista detalhada de itens comprados, custo de cada item, custo total a pagar (como um recibo) ou calcular juros sobre hipotecas e imprimir parcelas ao longo de um período
- Educação: encontrar e imprimir o aluno com a maior pontuação em cada disciplina ou verificar as entradas do usuário em relação à entrada esperada (como pontuação de teste); encontrar a nota média da disciplina (MSG) para uma turma

- Científico/Matemática: determinar se um número de entrada é ímpar ou par e se um número é divisível por outro número
- Entretenimento: um jogo de adivinhação de números ou qualquer jogo que envolva manipulação de texto

3.1.2 Resolver problemas decompondo-os em partes menores e gerando

3.1.3 Identificar elementos comuns entre problemas semelhantes e desenvolver

w Decomposição é uma forma de pensar sobre problemas, algoritmos e sistemas em termos de suas partes. As

partes podem então ser compreendidas, resolvidas, desenvolvidas e avaliadas separadamente.

Essa abordagem facilita a resolução de

problemas complexos e o design de grandes sistemas.

x Generalização é uma forma de resolver rapidamente novos problemas anteriores que foram resolvidos. Ou seja,

o algoritmo que funciona para um problema específico anterior é adaptado para resolver uma classe inteira de problemas semelhantes

através da identificação de padrões e pontos em comum em problemas, processos, soluções ou dados.

16

3.2 Design de Algoritmos

Esta unidade de estudo trata da interpretação e compreensão de algoritmos; da correção e escrita de algoritmos para problemas dados; bem como do refinamento de algoritmos. A aprendizagem dos alunos será aprimorada através de exercícios escritos, discussões em sala de aula e apresentações.

Ref. Resultado de Aprendizagem

Os alunos devem ser capazes de

3.2.1 Realizar uma execução simulada (dry run) de um conjunto de passos para determinar seu propósito e/ou saída.

3.2.2 Produzir tabelas de rastreamento para mostrar os valores das variáveis em cada etapa de um conjunto de passos.

3.2.3 Localizar erros lógicos em um algoritmo e corrigi-lo ou modificá-lo para remover os erros ou para alterações na especificação da tarefa.

3.2.4 Produzir um algoritmo, em pseudo-código ou diagramaticamente, para resolver um problema.

17

Módulo IV: Programação

Este módulo trata da aplicação e desenvolvimento do pensamento lógico e do raciocínio, bem

como das habilidades de resolução de problemas através do design e desenvolvimento de soluções de software usando linguagens de programação. Um algoritmo descreve uma solução independente de uma linguagem de programação, enquanto uma linguagem de programação ilustra a solução que é viável em um dispositivo de computação. Os alunos testarão se seus algoritmos funcionam conforme o planejado quando executarem as soluções de programação. Há duas unidades de estudo:

4.1 Desenvolvimento de Programas

4.2 Teste de Programas

4.1 Desenvolvimento de Programas

Esta unidade de estudo abrange o desenvolvimento de soluções de programação (codificação) para situações problemas simples. Os alunos reforçarão sua compreensão através de trabalho prático. Python será a linguagem de programação recomendada.

Ref. Resultado de Aprendizagem

Os alunos devem ser capazes de

4.1.1 Compreender e descrever as etapas no desenvolvimento de um programa: coletar requisitos, planejar soluções, escrever código, testar e refinar código, implementar código.

4.1.2 Compreender e usar estruturas de sequência, seleção e iteração para criar um programa.

4.1.3 Utilizar e justificar o uso de variáveis, constantes e listas simples (vetores 1D) em diferentes contextos de problema.

4.1.4 Compreender, utilizar e justificar o uso de diferentes tipos de dados e funções incorporadas em programas.

4.1.5 Produzir solução de programação para um problema dado para:

- encontrar valor min/max em uma lista
- encontrar a média de uma lista de valores numéricos
- procurar um item em uma lista e relatar o resultado da pesquisa
- encontrar dígitos de verificação
- encontrar o comprimento de uma cadeia de caracteres
- extrair caracteres necessários de uma cadeia de caracteres

18

4.2 Teste de Programas

Esta unidade de estudo abrange o teste e o refinamento de programas com base nos resultados dos testes. Os alunos aprenderão o uso apropriado de casos de teste e que tipo de testagem é necessária e/ou suficiente. Os alunos reforçarão sua aprendizagem e compreensão através de trabalho prático. Python será a linguagem de programação recomendada para a codificação.

Ref Resultado de Aprendizagem

Os alunos devem ser capazes de

4.2.1 Identificar e justificar o uso de técnicas de validação de dados.

4.2.2 Validar dados de entrada para aceitação realizando

- verificação de comprimento,
- verificação de intervalo,
- verificação de presença, e
- verificação de formato

4.2.3 Projetar casos de teste apropriados para cobrir condições normais, de erro e de limite, e especificar o que está sendo testado para cada caso de teste.

4.2.4 Compreender e descrever os tipos de erros de programa: sintaxe, lógica e tempo de execução; e explicar por que eles ocorrem.

4.2.5 Explicar como os tradutores são usados para detectar erros de sintaxe e indicar a diferença entre tradutores intérpretes e compiladores.

4.2.6 Compreender e aplicar técnicas de depuração para isolar/identificar e depurar erros de programa: uso de declarações de impressão intermitentes, percorrendo um programa, testando o programa em pequenos blocos ou por partes, e usando ajuda de depuração incorporada.

19

SEÇÃO 3:

PEDAGOGIA

Aprendizagem Aplicada

Aprender Fazendo

20

3. PEDAGOGIA

Aprendizagem Aplicada

Como disciplina aplicada, há maior ênfase na aprendizagem prática no contexto do mundo real. As pedagogias e atividades utilizadas devem, portanto, ser adequadas para a aprendizagem aplicada. Além disso, pretende-se que os alunos recebam oportunidades de adquirir Competências do Século XXI por meio da aprendizagem aplicada.

Competências do Século XXI (CC21)

A Tabela 3.1 ilustra como o currículo de Computação de Nível-O está alinhado com os Padrões e Referenciais para CC21. Para ilustrar, os alunos desenvolverão habilidades de pensamento crítico e inventivo (2.1d) quando lhes for atribuída a tarefa de projetar um programa para calcular recibos de pontos de venda. Os alunos precisarão avaliar informações e requisitos de processos, e analisar custos e benefícios antes de aplicar raciocínio sólido e tomada de decisão para propor uma solução viável em relação ao hardware e software a serem adquiridos.

Tabela 3.1: Alinhamento com Padrões e Referenciais de CC21

Competências e Atitudes em Computação

Nível-O Referências de Competências do Século XXI
(Até o final do 4º/5º ano do Ensino Médio)

Computador como Ciência Pensamento Crítico e Inventivo (PCI)

Capacidade de fazer brainstorming de ideias para soluções de problemas. 1.1d: O aluno é capaz de gerar ideias e explorar diferentes caminhos que levam a soluções.

Capacidade de aplicar o pensamento computacional através de:

- Sintetizar conhecimentos e habilidades das cinco principais áreas da Ciência da Computação; e
- Aplicar raciocínio formal e pensamento sistêmico na análise, projeto e implementação de soluções computacionais.

2.1d: O aluno é capaz de usar evidências e adotar diferentes pontos de vista para explicar seu raciocínio e decisões, tendo considerado as implicações da relação entre diferentes pontos de vista.

Capacidade de depurar e refinar programas de computador (pensamento computacional). 2.2d: O aluno é capaz de suspender o julgamento, reavaliar conclusões e considerar alternativas para refinar seus pensamentos, atitudes, comportamento e ações.

Capacidade de: 3.1d: O aluno é capaz de identificar elementos essenciais de tarefas complexas, manter o foco nelas, assumir diversos papéis

21

- analisar e simplificar problemas em tarefas gerenciáveis (pensamento analítico);
- persistir no desenvolvimento de soluções de computador para os problemas (resiliência); e
- avaliar soluções usando casos de teste (pensamento avaliativo).

e perseverar quando encontram dificuldades e desafios inesperados.

Computador como Ferramenta Habilidades de Informação e Comunicação (HIC)

Habilidade de explicar e comunicar soluções de programação. 1.2d: O aluno é capaz de usar informações e ideias desenvolvidas coletivamente com outros para criar novas informações, produtos e/ou soluções.

Habilidade de ser engenhoso na busca de informações pertinentes necessárias para resolver o problema. 2.1d: O aluno é capaz de integrar informações de uma variedade de fontes para completar uma tarefa.

Computador na Sociedade Alfabetização Cívica, Consciência Global e Habilidades Interculturais (ACGC)

Habilidade de compreender proteção de dados, questões de direitos autorais e direitos de propriedade intelectual. 1.1d: O aluno é capaz de discutir questões que afetam a cultura, o desenvolvimento socioeconômico, a governança, o futuro e a identidade de Singapura, e usar evidências para apoiar seus pontos de vista.

Adotar práticas éticas no ciberespaço e nas mídias sociais.

Situações Autênticas

Estudos de caso reais sobre proteção de dados, por exemplo, serão usados para que os alunos investiguem a necessidade de privacidade e integridade dos dados, e para comparar/contrastar as diferentes medidas de proteção de dados. Os alunos perceberão os perigos e compreenderão os erros cometidos por outros através de transações online ou compartilhamento de informações pela internet; e aprenderão a ser perspicazes e cuidadosos com suas informações pessoais, ao mesmo tempo em que adotam práticas éticas no ciberespaço.

22

Aprender Fazendo

Recomenda-se uma ampla gama de pedagogias para o ensino de Computação Nível-O. Por exemplo, utilizando uma abordagem de sala de aula invertida, os alunos podem assistir a um vídeo (em seu próprio tempo) sobre as considerações de design e a configuração real de uma rede sem fio simples

em um ambiente doméstico ou empresarial. Os alunos então configurarão uma rede simples fisicamente na sala de aula de informática ou usarão um simulador baseado na web disponível no Espaço de Aprendizagem do Aluno e aprenderão fazendo. Embora as pedagogias aplicadas dependam da natureza do tópico, uma abordagem baseada em problemas pode ser utilizada.

23

SEÇÃO 4:

AVALIAÇÃO

Avaliação Escolar

Exame Nacional

24

4. AVALIAÇÃO

Avaliação Escolar

Avaliação para a Aprendizagem

Os alunos têm a oportunidade de uma experiência de aprendizagem profunda e enriquecida através do uso de ambientes de aprendizagem online no Espaço de Aprendizagem do Aluno ou na internet. Eles podem monitorar e autorregular seu ritmo e progresso de aprendizagem. Eles recebem feedback imediato desses ambientes de aprendizagem sobre erros em suas soluções de programação e se os resultados de suas soluções de programação são os pretendidos.

As seis semanas reservadas para o projeto de programação permitirão que os alunos colaborem e aprendam uns com os outros. Além da avaliação para a aprendizagem, a avaliação da aprendizagem também ocorre quando os alunos aplicam os conceitos e habilidades de programação que aprenderam ao projeto. O pensamento computacional também será desenvolvido e, se o projeto for realizado em grupo, os alunos também aprenderão a explorar diferentes caminhos para soluções e aprimorar as habilidades de comunicação.

Avaliação da Aprendizagem

O Espaço de Aprendizagem do Aluno também permitirá que os alunos avaliem seu aprendizado, bem como aprendam com as tarefas de avaliação. A avaliação escolar também pode consistir em avaliações escritas e práticas cronometradas da compreensão dos alunos dos quatro módulos do programa. As questões devem testar mais a compreensão e aplicação dos conceitos e habilidades aprendidas pelos alunos, e menos a memorização de informações. A prova escrita pode compreender questões estruturadas curtas de valores variáveis de notas.

Um modelo de prova baseada em laboratório pode consistir em uma tarefa de planilha e uma ou duas tarefas de programação. Alternativamente, a prova pode consistir em uma série de planejamento e tarefas desenvolvidas progressivamente, culminando em uma solução de programação final.

O formato das provas de avaliação também pode ser modelado de acordo com o formato dos exames nacionais. As notas da avaliação escolar podem ser usadas para relatar o desempenho dos alunos no final dos semestres.

25

Exame Nacional

Objetivos da Avaliação

O exame avaliará:

- a) Conhecimento e compreensão de tecnologia e sistemas básicos de computação, conceitos, algoritmos, técnicas e ferramentas;
- b) Aplicação de conhecimento e compreensão para analisar e resolver problemas de computação;
- c) Desenvolvimento, teste e refinamento de soluções utilizando aplicativos de software e/ou linguagens de programação apropriadas.

A ponderação para cada objetivo de avaliação é mostrada na Tabela 4.1.

Tabela 4.1 Grade de Especificação

Ref.	Objetivo da Avaliação	Ponderação
	na Prova 1	
(a)	Conhecimento e compreensão	~ 35%
(b)	Aplicação	~ 25%
(c)	Desenvolvimento, teste e refinamento	~10%
	TOTAL	70%

Ponderação	Ponderação
na Prova 2	Geral
~5%	40%
~5%	30%
~20%	30%
30%	100%

Os alunos podem manusear e processar dados em sistemas de computador, bem como a necessidade de ser ético ao lidar com dados. Eles demonstrarão técnicas de resolução de problemas através da análise e escrita de soluções de programação para uma variedade de problemas de computação em negócios, educação, matemática e ciência. Os alunos serão capazes de demonstrar pensamento computacional através do design e desenvolvimento de soluções de programação.

Modo de Exame

Todos os candidatos farão as Provas 1 e 2. Todas as questões são obrigatórias em ambas as

provas.

Prova 1 (Exame escrito, 2 horas)

Esta prova testa o conhecimento, a compreensão e a aplicação de conceitos e habilidades em todos os quatro módulos. As questões consistem em uma mistura de:

- questões de resposta curta
- questões de associação
- preenchimento de lacunas
- questões estruturadas

Esta prova vale 70% da pontuação total e é avaliada em 80 pontos.

26

Prova 2 (Exame em Laboratório, 2 horas e 30 minutos)

Esta prova, realizada com o uso de um computador, planilhas e software, avalia o Módulo 1 (Unidade 1.1: Gerenciamento de Dados) e o Módulo 4 (Programação). Quatro questões estruturadas serão baseadas nos seguintes tópicos:

- Uso de funções e recursos de planilhas
- Refinamento de programa
- Depuração de programa
- Desenvolvimento de programa com no máximo 40 linhas

O desenvolvimento do programa valerá 20 pontos. As três questões restantes têm uma média de 10 pontos.

Os candidatos deverão submeter cópias digitais do trabalho exigido para correção. O tempo alocado inclui tempo para salvar o trabalho exigido no computador dos candidatos. Esta prova vale 30% da pontuação total e é avaliada em 50 pontos.

A Tabela 4.2 resume os formatos das duas provas obrigatórias.

Tabela 4.2: Formato das Provas

Prova	Modo	Duração	Ponderação	Notas	Formato	Módulos
		Avaliados				
1	Escrita	2 horas	70%	80	Uma mistura de	Todos os
			• Perguntas de resposta curta • Perguntas de correspondência • Passagem com lacunas • Perguntas estruturadas		módulos	quatro
2	Lab-baseada prática	2 horas 30 minutos	30%	50	4 questões estruturadas sobre	Unidade 1.1
			• Uso de funções de	Gestão de	Dados do	

Módulo 4:

Programação

- Refinamento de programa
- Depuração de programa
- Desenvolvimento de programa com no máximo 40 linhas de código

O desenvolvimento do programa valerá 20 notas.

As três questões restantes terão uma média de 10 notas por questão.

y O centro será obrigado a declarar a(s) linguagem(ns) de programação antes que o centro inicie o ensino do curso para a turma que fará o exame.

27

Uso de calculadora

Uma calculadora aprovada pode ser usada nas Provas 1 e 2.

Lista de Glossário

Os candidatos receberão no exame uma lista de glossário de sintaxe de programação. O glossário está no Anexo A.

Infraestrutura do Centro para Exame em Laboratório

O Centro garantirá instalações adequadas de hardware e software para apoiar o exame de seus candidatos para a Prova 2, que será administrada em no máximo dois turnos no dia do exame. Cada candidato deve ter o uso exclusivo de um computador pessoal para fins de exame. Os candidatos devem ter acesso a software de aplicativo de planilha e software de linguagem de programação. O Centro será obrigado a declarar o nome e o número da versão do software a ser usado para a turma que fará o exame pelo menos dois anos antes de a turma fazer o exame.

28

ANEXO A

Referência Rápida para Python

Esta referência rápida mostra alguns exemplos de construções da linguagem Python. A linguagem Python completa não se limita a esses exemplos.

1. Identificadores

Ao nomear funções, variáveis e módulos, as seguintes regras devem ser observadas:

-

Os nomes devem começar com o caractere 'a' - 'z' ou 'A' - 'Z' ou '_' e seguidos por caracteres alfanuméricos ou '_'

-

-

Palavras reservadas não devem ser usadas.

Identificadores definidos pelo usuário diferenciam maiúsculas e minúsculas.

2. Comentários e Strings de Documentação

Este é um comentário

"""

Esta é uma string de documentação em várias linhas

"""

3. Entrada/Saída

```
print ("Esta é uma string")
```

```
s = input ("Instruções para solicitar a entrada de dados.")
```

4. Importação

```
import <módulo>
```

ex: import math

5. Tipo de Dados

Tipo de Dados	Notas
---------------	-------

int	inteiro
-----	---------

float	número real
-------	-------------

bool	booleano
------	----------

str	string (imutável)
-----	-------------------

list	série de valores
------	------------------

6. Atribuição

Instrução de Atribuição	Notas
-------------------------	-------

a = 1	inteiro
-------	---------

b = c	variável
-------	----------

d = "Esta é uma string"	string
-------------------------	--------

minhalista = [1, 2, 3, 4, 5]	lista ou vetor
------------------------------	----------------

7. Operadores Aritméticos

Operador	Notas
+, -	mais, menos
*, /	multiplicar, dividir
%	resto ou módulo
**	exponencial ou potência
//	quociente da divisão inteira

8. Operadores Relacionais

Operador	Notas
==	igualdade
!=	diferente de
>, >=	maior que, maior ou igual a
<, <=	menor que, menor ou igual a

9. Expressão Booleana

Expressão Booleana	Notas
a e b	e lógico
a ou b	ou lógico
não a	não lógico

10. Iteração

loop while loop for

enquanto condição(ões):
<instrução(ões)>

para i em range(n):
 <instrução(ões)>

para registro em
registros:
 <instrução(ões)>

29

ANEXO A

11. Seleção

Tipo 1 Tipo 2 Tipo 3

se condição(ões):
 <instrução(ões)>

se condição(ões):
 <instrução(ões)>

senão:

<instrução(ões)>

se condição(ões):

<instrução(ões)>

senão se condição(ões):

<instrução(ões)>

senão:

<instrução(ões)>

12. Funções Embutidas

(a) Funções básicas

abs() chr() float() input() int()
ord() print() range() round() str()
format()

(b) Funções matemáticas

ceil() exp() fabs() floor() log()
max() min() pow() sqrt() trunc()

(c) Funções de string

endswith() find() isalnum() isalpha() isdigit()
islower() isspace() isupper() len() lower()
startswith() upper()

13. Palavras Reservadas

Palavras reservadas não podem ser usadas como identificadores. Elas fazem parte da sintaxe da linguagem.

False None True and as
assert break class continue def
del elif else except finally
for from global if import
in is lambda nonlocal not
or pass raise return try
while with yield

30

ANEXO B

Referência Rápida para Fluxogramas

Esta referência rápida mostra quatro símbolos comuns usados em fluxogramas de programa e

fornece um
guia padrão para a construção de fluxogramas de programa.

1. Símbolos comuns

Terminador Processo Dados Decisão

2. Construindo fluxogramas

(a) Comece e termine com UM símbolo de terminador

Iniciar

O

O

O

Fim

Iniciar

O

O

O

Fim

X

Fim

(b) Ter uma única entrada e uma única saída (exceto para os símbolos de terminador e de decisão)

V

V

X

X

X

Existem três saídas possíveis
para o símbolo de decisão.

31

ANEXO B

(c) Linhas de fluxo não devem se cruzar

X

Alguns usavam uma dobra na linha de fluxo, mas isso não
é recomendado para nossos alunos.

32

ANEXO C

Glossário de Termos

Os alunos devem estar familiarizados com os seguintes termos nos resultados de aprendizagem, e esses termos serão usados de forma semelhante nas questões do exame.

Termo	Resultados de aprendizagem	Definição
Aplicar	4.2.6	Executar ou usar técnicas ou procedimentos comuns.
Comparar	1.3.4	Dar um relato dos aspectos positivos e negativos, ou vantagens e desvantagens de dois itens.
Comparar e contrastar	2.2.4	Mapear, combinar. Dar um relato das semelhanças e diferenças de dois itens.
Construir	2.1.3, 2.1.4	Produzir, criar. Juntar elementos para formar um todo coerente; reorganizar em um novo padrão ou estrutura.
Converter	1.2.2	Transformar, expressar de outra forma ou maneira.
Corrigir	3.2.3	Retificar, fazer alterações para melhoria.
Definir	3.1.1	Redefinir, dizer de outra forma e dar mais detalhes ou informações relacionadas.
Descrever	1.2.2, 1.2.3, 1.3.3, 2.1.1, 2.2.2, 2.2.3, 4.1.1, 4.2.4	Dar um relato detalhado ou mais informações sobre o "o quê" e o "como".
Projetar	2.1.4, 2.2.3, 4.2.3	Planejar, pensamento visível, evidência de ideias, por exemplo, um artefato, um programa de computador, um resultado, um diagrama, um desenho ou um modelo.
Explicar	1.3.4, 2.2.1, 2.2.5, 4.2.4, 4.2.5	Construir modelos. Inclui elementos de análise. Dar um relato detalhado ou mais informações sobre o "onde", "como" e "por que".
Avaliar	2.1.2	Fazer julgamentos baseados em critérios e padrões.
Identificar	2.2.1, 3.1.3, 4.2.1	Reconhecer, nomear. Capacidade de diferenciar e discriminar.
Justificar	4.1.3, 4.1.4, 4.2.1	Dar razões ou evidências para apoiar uma ação ou decisão.
Localizar	3.2.3	Encontrar.
Modificar	3.2.3	Fazer alterações.
Executar	3.2.1	Fazer, realizar.

As definições em itálico são baseadas na Taxonomia Revisada de Bloom (Anderson e Krathwohl, 2001)¹ p em um folheto da Iowa State University, Centro de Aprendizagem e Ensino:

- Lembrar - recuperar conhecimento relevante da memória de longo prazo
- Compreender – construir significado a partir de mensagens instrucionais, incluindo comunicação oral, escrita e gráfica
- Aplicar - executar ou usar um procedimento em uma determinada situação
- Analisar - quebrar o material em partes constituintes e determinar como as partes se relacionam umas com as outras e com uma estrutura ou propósito geral
- Avaliar - fazer julgamentos com base em critérios e padrões
- Criar – juntar elementos para formar um todo coerente; reorganizar em um novo padrão ou estrutura

10 Anderson, L.W. (Ed.), Krathwohl, D.R. (Ed.), Airasian, P.W., Cruikshank, K.A., Maye, R.E., Pintrich, P.R.,
 Raths, J., & Wittrock, M.C. (2001). A taxonomy for learning, teaching and assessing: a revision of Bloom's
 Taxonomy of Educational Objectives (Complete edition). Nova York: Longman.